

Software Requirements Specification (SRS)

DevCommandCenter — Unified Developer Context, Deployment & Identity Manager

Document Ver: 1.0.0

Status: Approved Baseline

Target Platform: Web (Next.js) & Date: September 2026
Android

1. Introduction

1.1 Purpose

The purpose of this document is to provide a complete and authoritative Software Requirements Specification (SRS) for **DevCommandCenter**. DevCommandCenter is an engineering tool designed to solve multi-stack developer context fragmentation. It tracks dedicated project identities, source repositories, frontend/backend hosting environments, local execution runbooks, and test credentials with zero-knowledge encryption and cross-platform synchronization.

1.2 Scope

The system encompasses a responsive Web Application (Next.js) and an Android Mobile Application (Flutter/Kotlin) connected to a managed Supabase backend (PostgreSQL + Realtime Sync + Auth). The Android client strictly enforces device-level screen lock and hardware-backed biometric verification before decrypting sensitive credentials. The system operates on a zero-cloud-cost tier.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
AES-GCM-256	Advanced Encryption Standard in Galois/Counter Mode with 256-bit keys, providing authenticated encryption and data integrity.
PBKDF2	Password-Based Key Derivation Function 2 (with HMAC-SHA256, 100,000+ iterations), used to derive encryption keys from user passphrases.
RLS	Row Level Security — PostgreSQL security primitive ensuring multi-tenant data isolation at the database layer.
Keystore	Android hardware-backed cryptographic container preventing key extraction even with root permissions.
Runbook	Executable technical instructions documenting runtime prerequisites, environment configs, and startup commands.

2. Overall System Description

2.1 Product Perspective & System Architecture

DevCommandCenter follows a decoupled client-server architecture with client-side Zero-Knowledge encryption:

```
+-----+ | DevCommandCenter
Architecture | +-----+ | [
Web Client: Next.js + Tailwind ] [ Mobile Client: Flutter / Android ] | | - Master Passphrase In-
Memory Cache - Android Keystore Hardware Binding | | - Web Crypto API (SubtleCrypto) - BiometricPrompt
(Fingerprint/PIN) | +-----+
| HTTPS / WSS (TLS 1.3) v
+-----+ | Supabase Managed
Cloud Backend | | - Supabase Auth (JWT Token Verification) | | - PostgreSQL 15+ (Row Level Security
enabled on all tables) | | - Realtime WebSockets Engine (Change Data Capture replication) |
+-----+ | * Note: Sensitive
payload fields (passwords, tokens) are encrypted on-device | | prior to transport. The database never
receives plain-text secrets. |
+-----+
```

2.2 User Characteristics

The primary end-user is a computer engineering student, software developer, or independent technical architect who manages multiple software repositories, diverse cloud infrastructure providers (Vercel, Render, Railway, AWS, GCP), and multiple dedicated email accounts.

2.3 Assumptions & Operational Constraints

- **Zero-Dollar Operational Budget:** The deployment must fit within standard free tiers (Supabase Free Tier, Vercel Hobby, local direct APK installation).
- **Client Environment:** Android devices must run Android 9.0 (API Level 28) or higher to support modern biometric and Keystore APIs. Modern web browsers must support the standard Web Cryptography API (SubtleCrypto).
- **Network Connectivity:** Online connection is required for cloud synchronization; local read cache provides offline access to previously fetched runbooks.

3. Specific Functional Requirements

3.1 Module 1: Dedicated Email & Identity Management

Req ID	Requirement	Technical Specification & Verification
FR-ACC-01	Account Creation & Tracking	The system shall record email addresses, service providers (Gmail, Outlook, custom), target purpose, creation date, and linked recovery identifiers (backup email, recovery phone suffix).
FR-ACC-02	Zero-Knowledge Password Vault	Passwords, app-specific SMTP tokens, and 2FA recovery backup codes must be encrypted using client-side AES-GCM-256 before write operations to Supabase.
FR-ACC-03	Recovery Association	The system shall allow linking each account to its parent recovery email to prevent account lockouts.

3.2 Module 2: Project Deployment & Repository Tracking

Req ID	Requirement	Technical Specification & Verification
FR-PRJ-01	Source Control Metadata	Track repository URL, GitHub/GitLab profile handle, default working branch, and associated identity account.
FR-PRJ-02	Dual-Tier Deployment Mapping	

Req ID	Requirement	Technical Specification & Verification
		Independently log Frontend and Backend deployment details: platform name (e.g., Vercel, Railway), deployment account email, live production URL, and administrative console links.
FR-PRJ-03	Test Credentials Store	Maintain key-value pairs or structured credentials (test accounts, API mock tokens) encrypted with client-side AES-GCM-256.
FR-PRJ-04	Local Execution Runbook	Store Markdown-compatible text detailing runtime prerequisites (e.g., Python 3.11, Node 20), <code>.env`</code> variable keys, and CLI start sequences ( <code>`docker compose up`</code> , <code>`npm run dev`</code>).

3.3 Module 3: Android Biometric & Security Module

Req ID	Requirement	Technical Specification & Verification
FR-SEC-01	Hardware-Backed Auth Gating	The Android app shall gate decryption operations behind <code>BiometricPrompt</code> with <code>BIOMETRIC_STRONG</code> or <code>DEVICE_CREDENTIAL</code> fallback.
FR-SEC-02	Screen Leak Prevention	Set <code>WindowManager.LayoutParams.FLAG_SECURE</code> to block screenshots and task-switcher previews of decrypted credentials.
FR-SEC-03	In-Memory Auto-Wipe	Decrypted secrets must be cleared from heap memory on app pause (<code>`onPause`</code>) or after 5 minutes of idle time.

3.4 Module 4: Realtime Synchronization & Portability

Req ID	Requirement	Technical Specification & Verification
FR-SYN-01	Bi-directional Realtime Sync	Changes made on the web app must reflect on the Android app within <code>< 1000ms</code> using Supabase Realtime WebSocket subscriptions.
FR-SYN-02	One-Click Data Portability	The system shall provide a full client-side JSON export of all projects and accounts, allowing backup without vendor lock-in.

4. Database Schema Specification

The database layer operates on PostgreSQL within Supabase. Strict Row Level Security (RLS) ensures that unauthenticated clients or external users cannot read or write records.

```

-- ACCOUNTS ENTITY
CREATE TABLE public.accounts (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users NOT NULL DEFAULT auth.uid(),
  email TEXT NOT NULL,
  provider TEXT DEFAULT 'Gmail',
  purpose TEXT,
  recovery_email TEXT,
  encrypted_password TEXT,
  encrypted_notes TEXT,
  created_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- PROJECTS ENTITY
CREATE TABLE public.projects (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users NOT NULL DEFAULT auth.uid(),
  name TEXT NOT NULL,
  description TEXT,
  tags TEXT[],
  git_repo_url TEXT,
  git_account_email TEXT,
  frontend_platform TEXT,
  frontend_account_email TEXT,
  frontend_url TEXT,
  backend_platform TEXT,
  backend_account_email TEXT,
  backend_url TEXT,
  local_runbook TEXT,
  encrypted_test_credentials TEXT,
  linked_account_id UUID REFERENCES public.accounts(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- ROW LEVEL SECURITY ENFORCEMENT
ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users access own accounts" ON public.accounts FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users access own projects" ON public.projects FOR ALL USING (auth.uid() = user_id);

```

5. Non-Functional Requirements (NFRs)

5.1 Security Requirements

- **Transport Security:** All client-server communications must use TLS 1.3 encryption. Plain HTTP is strictly rejected.
- **Zero-Knowledge Architecture:** Master passphrases are never sent over the network. Password hashes and encryption keys are generated client-side via PBKDF2 (100,000 iterations, SHA-256).
- **Key Isolation:** Mobile cryptographic keys are managed by the Android Keystore system with user presence verification enforced by the operating system.

5.2 Performance & Scalability Requirements

- **Initial View Latency:** Project index view must render in < 1.2s on standard 4G mobile and broadband web connections.

- **Decryption Overhead:** Decryption of credential blocks must execute within < 50ms upon successful biometric confirmation.
- **Database Capacity:** The 500 MB free PostgreSQL tier can store over 50,000 project and account records under standard JSON and text sizes.

5.3 Usability & UX Requirements

- **One-Tap Clipboard Copy:** Every repository link, local terminal command, live URL, and credential field must provide a single-touch copy-to-clipboard action with visual toast feedback.
- **Offline Read-Only Resilience:** The mobile app must cache the latest project list locally (encrypted) so developers can check local runbooks even without internet connectivity.

6. Verification, Validation & Test Matrix

Test ID	Target Requirement	Test Condition & Methodology	Expected Result
TC-SEC-01	Zero-Knowledge Storage	Inspect raw rows in Supabase Table Editor after adding a project credential.	Values must be ciphertext strings (`U2FsdGVk...`); no plaintext passwords present.
TC-MOB-01	Biometric Authentication Gate	Attempt to view encrypted credentials without completing biometric or device PIN challenge.	Decryption fails; access is denied until OS confirms authentication.
TC-SYN-01	Cross-Platform Sync	Add a new runbook command on the Web app while the Android app is open on the same project view.	Android view updates automatically within 1.0 second via WebSocket subscription.
TC-RLS-01	Multi-Tenant Isolation	Authenticate as User B and query User A's project UUID using the public anon key.	Query returns empty array or HTTP 403 Forbidden due to Postgres RLS.

Document Sign-off: This Software Requirements Specification constitutes the baseline development contract for DevCommandCenter. Any deviations, schema expansions, or protocol adjustments must follow standard versioning and change control protocols.